

Introducción a la Ingeniería de Requerimientos

Clase 2 — Dificultades Esenciales, Modelos y Stakeholders

Material teórico completo — texto, imágenes y videos complementarios

Introducción

En la clase de hoy verá una aproximación a la ingeniería de requerimientos, sus dificultades, los stakeholders, el problema de la comunicación, entre otros conceptos. Comenzaremos por ver algunas dificultades esenciales del software.

- Problemática de la representación / modelo / descripción.
- Tipos de modelos.
- Dificultades para comprender los requerimientos.
- Stakeholders.
- El punto de vista de los diferentes stakeholders.
- El gap semántico.

Las dificultades esenciales del software

Hay que distinguir dos tipos de dificultades en el software: las **esenciales**, inherentes a su naturaleza, y las **accidentales**, que ofrece (actualmente) su producción pero que no son inherentes al software (son propias del desarrollo).

La esencia de una entidad de software es que es una construcción de conceptos interconectados: conjuntos de datos, relaciones entre ítems de datos, algoritmos e invocaciones de funciones. Esta esencia es abstracta y no varía con las distintas representaciones.

Propiedades inherentes a la esencia del software:

- Complejidad
- Conformidad (conformity)
- Modificabilidad
- Invisibilidad

Complejidad

Las construcciones de software probablemente son más complejas que cualquier otra construcción humana, incluyendo las computadoras. "Escalar" el software no consiste en repetir los mismos elementos en una escala mayor.

De la complejidad esencial y de su incremento no lineal con el tamaño se derivan importantes consecuencias:

- Problemas clásicos de la producción de software: la dificultad de comunicación.
- De la complejidad de la estructura deriva la dificultad de extender los programas a nuevas funciones y los estados no visualizados.
- Problemas de administración de los proyectos: hace difícil la visión global, impidiendo la integración conceptual.

- Crea la tremenda necesidad de aprendizaje y comprensión que convierte a la rotación de personal en un desastre.

Conformidad

Parte importante de la complejidad del software es de origen arbitrario, derivada de las instituciones y sistemas a los que su interfaz debe conformarse. El software debe conformar porque:

- Muchas veces es el último en llegar a la escena.
- Otras veces se lo percibe como el que tiene mayor capacidad de conformar interfaces.

Modificabilidad

El software está constantemente bajo la presión del cambio; en los productos manufacturados los cambios sustanciales se hacen en series posteriores. Razones de cambios en el software:

- El software corporiza funciones, y estas son las que mayor presión tienen por el cambio.
- El software puede cambiarse muy fácilmente, es muy maleable.

Todo el software exitoso es objeto de cambios, porque:

- Si funciona bien se le añaden funciones a las originales.
- Sobrevive a la máquina para la que se diseñó originalmente.

En resumen:

- El software está embebido en una matriz cultural de: aplicaciones, usuarios, leyes y máquinas.
- Estas cambian continuamente y sus cambios fuerzan cambios sobre el producto de software.

Invisibilidad

La realidad del software no está sumergida en el espacio, luego no tiene representaciones geométricas en el sentido en que la tierra tiene mapas, los chips tienen diagramas y las computadoras tienen diagramas de interconexión.

El software tiene varios diagramas que pueden representar:

- Flujo de control
- Flujo de datos
- Patrones de dependencia
- Secuencia de tiempo
- Relaciones nombre-espacio

Se progresó en restringir y simplificar las estructuras del software, pero permanece no visualizable, por lo que no permite que la mente use algunas de sus herramientas más poderosas y se dificulta la comunicación entre varias mentes.

Impacto de las dificultades esenciales

La complejidad, la conformidad, la modificabilidad y la invisibilidad impactan en:

- Cómo entendemos lo que nos piden que hagamos.
- Qué tenemos que hacer.
- La volatilidad de lo que nos piden.
- La imposibilidad de visualizar el producto.

Video complementario sugerido para ampliar estos conceptos: ver sección "Videos complementarios" al final del documento (video 1).

Problemática de la representación, el modelo y la descripción

Existen dificultades esenciales a la hora de desarrollar software, así como la problemática asociada a la representación, el modelo y la descripción; la dificultad para comprender los requerimientos; y la comunicación y relación con los stakeholders. Existe una amplia variedad de conceptos de modelo. Un modelo puede ser:

- Una copia pequeña y exacta de algo (por ejemplo, el aeromodelismo). Una copia no es un modelo: no tiene las ventajas de un modelo (ej. reducción de costos) ni sus desventajas (ej. imprecisión respecto al original).
- Una descripción o analogía utilizada para ayudar a visualizar algo que no puede ser observado directamente. Captura la motivación de la "simulación" o "descripción".
- Un patrón o figura de algo a construir. Apunta al plan de "construcción".
- Una proyección teórica de un sistema imaginario o posible. Esto se vincula con el interés de la Ingeniería de Software de utilizar modelos con un propósito de construcción; el "original" no tiene por qué existir o puede ser imaginario. Esta alternativa es la pertinente en nuestro campo.

¿Para qué, de qué, y cómo hacer modelos?

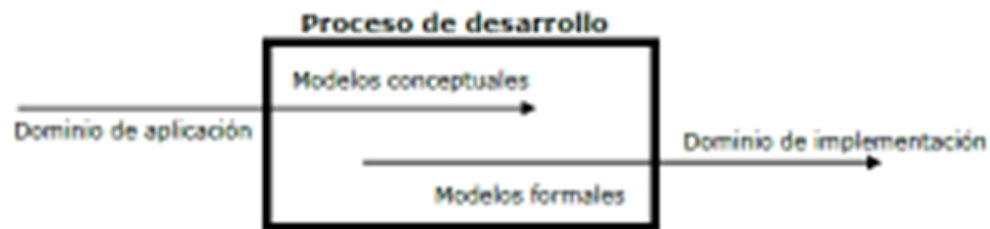
El modelo y la realidad son diferentes. Construimos modelos con motivos de representación y de construcción. Se construyen modelos de objetos que existen o no: a partir de la realidad construimos un modelo que la representa con el fin de entenderla (representación, existe); a partir de ese modelo construimos otro modelo que representa el artefacto que queremos construir (construcción, no existe).

Sustancia del proceso de desarrollo

Blum describe la esencia del proceso software en tres pasos: formular el modelo conceptual, que describe una solución al problema (el modelo conceptual carece del rigor y precisión del modelo formal); su traducción al modelo formal, que prescribe la solución implementada (en el modelo formal se pueden verificar las afirmaciones que contenga); y la construcción de una implementación que satisface el modelo formal.

Nótese que hablamos de desarrollo, no hemos considerado el proceso de obtención del conocimiento necesario para ese desarrollo.

■ ■



Proceso de desarrollo: del dominio de aplicación, a través de modelos conceptuales y modelos formales, hacia el dominio de implementación.

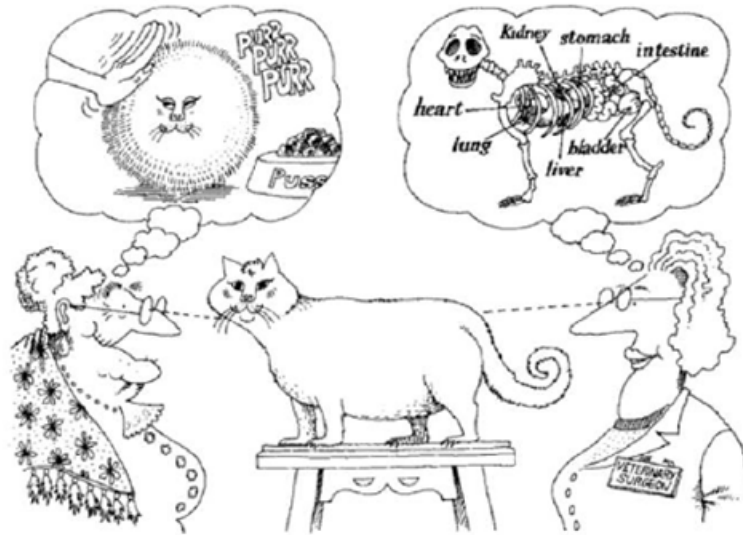
El desarrollo como secuencia de transformaciones

“[...] el proceso software siempre involucra transformaciones desde modelos mentales informales (que describen qué queremos hacer) hasta una implementación formal (que detalla cómo operará la solución prevista)” (Blum)

Necesitamos la abstracción para tratar con la complejidad (Rumbaugh):

- Examen selectivo de ciertos aspectos.
- Objetivo: aislar los aspectos importantes y suprimir los no importantes.
- La abstracción es guiada por un propósito (pues este define lo que no es importante).
- Es posible tener múltiples abstracciones de la misma cosa, dependiendo del propósito para el que se hacen.

El punto de vista del observador



“La abstracción enfoca las características esenciales de un objeto, relativas a las perspectiva del observador” (Booch)

“La abstracción enfoca las características esenciales de un objeto, relativas a la perspectiva del observador” (Booch): un mismo gato es visto de forma distinta según quién lo observa y con qué propósito.

Video complementario: ver sección "Videos complementarios" al final del documento (video 2).

La dificultad para obtener requerimientos

Brooks siempre presente...

“The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is as difficult as establishing the detailed technical requirements [...] No part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.” [Brooks, No Silver Bullets]

- Lo más difícil del trabajo conceptual.
- El impacto perjudicial del error.
- La dificultad de rectificar.

El problema principal es adquirir el conocimiento de los usuarios y otras fuentes. De allí se deriva: ¿dónde encontrar el conocimiento? Problemas en la captura del conocimiento:

- Formas no utilizables del conocimiento.
- Dificultades cuando se trata de un experto humano.

Los requerimientos:

- Sólo tienen sentido en el contexto organizacional.
- Son producto de la interacción usuario-técnico.

Usuarios:

- Falta de consenso entre usuarios.
- Tiempo limitado del usuario.
- Dificultades comunicacionales.
- Problemas del poder político.

Usuarios y desarrolladores:

- Problema de la comunicación.
- Diferencias de lenguaje.
- Diferencias de formación profesional.

Desarrolladores:

- Orientación a la solución.

Video complementario: ver sección "Videos complementarios" al final del documento (video 3).

Stakeholders

Son los individuos u organismos que ganan o pierden con algún cambio; son impactados positiva o negativamente, independientemente o no de su voluntad.

Loucopoulos (cap. 3): "the parties who have interest in the software system under development".
(Wieringa & Glinz, 2007): "A stakeholder is a person or organization who influences a system's requirements or who is impacted by that system".

Clasificación según su interés (Macaulay, 1996)

- Interesados en su construcción.
- Interés financiero.
- Interesados en su introducción y operación.
- Interesados en su uso.

El universo de los stakeholders:

- **Interesados en su construcción:** responsabilidad por el diseño y desarrollo. Incluye: project managers, diseñadores de software, expertos en comunicaciones.
- **Interés financiero:** responsabilidad por la compra o la venta. Incluye: analista de negocios, gerente de ventas, comprador.
- **Interesados en su introducción y operación:** responsabilidad por la implementación y el mantenimiento. Incluye: equipo de entrenamiento y soporte del usuario, ingenieros de instalación y mantenimiento, gerentes usuarios.

- **Interesados en su uso:** interés en su uso. Incluye: gerentes usuarios, toda clase de usuarios (directos e indirectos).

¿Cuáles son los objetivos de cada uno de estos grupos?

¿Cómo nos vemos el uno al otro?

Cómo vemos al usuario: ellos...

- No saben qué quieren.
- No pueden expresar qué quieren.
- Tienen muchas "necesidades" motivadas políticamente.
- Quieren todo ahora.
- No pueden priorizar necesidades.
- Quieren "primero yo", no "primero la empresa".
- Rechazan responsabilidad por el sistema.
- No están disponibles para proveer una definición de un sistema que funcione.
- No se comprometen con los proyectos de desarrollo.
- No desean el compromiso.

Cómo nos ve el usuario: nosotros...

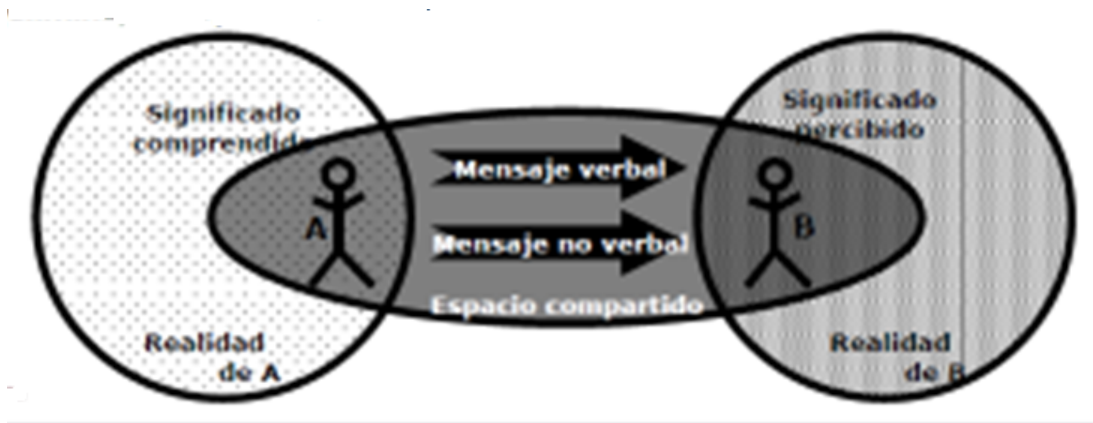
- No entendemos "el negocio".
- Manejamos la política de la empresa torpemente.
- Tratamos de explicarles cómo hacer su trabajo.
- No podemos traducir una definición de un sistema en un sistema exitoso.
- Decimos "no" todo el tiempo.
- Ponemos mucho énfasis en los tecnicismos.
- Siempre estamos por encima del presupuesto.
- Siempre estamos demorados.
- Reclamamos a los usuarios por tiempo y esfuerzo en detrimento de sus objetivos primarios.
- Establecemos estándares no realistas para la definición de requerimientos.
- No estamos disponibles para responder a cambios legítimos de necesidades.
- No podemos mantenernos en el cronograma.

La comunicación

El mensaje es completo: verbal y no verbal. Existen problemas en el emisor, el receptor y en el medio.

- **Emisor:** no emite lo que debe (omisión) o emite incorrectamente (error).
- **Receptor:** no recibe lo que debe (incompletitud) o recibe incorrectamente (error).

- **Medio:** perturba (elimina, incorpora, deforma).



Comunicación entre A y B: mensaje verbal y no verbal dentro de un espacio compartido, entre la realidad de A (significado comprendido) y la realidad de B (significado percibido).

La importancia de la comunicación

- Los stakeholders se comunican entre sí.
- La necesidad de comunicación entre los stakeholders es fuente de problemas de comunicación.
- El esfuerzo de desarrollo requiere comunicaciones con otras fuentes/receptores.
- Se amplifica el impacto de los problemas comunicacionales.

Gap semántico

Las aspiraciones, formación y experiencia de los usuarios y analistas difieren. Los primeros manejan su terminología orientada al dominio, en tanto que los analistas usan un vocabulario orientado a la computación. Hay una seria dificultad en establecer un esquema de comprensión entre el analista y el usuario, esto es, hacer que hablen el mismo lenguaje. Esto requiere reglas que ordenen la relación: el "contrato social" de los requerimientos.

Video complementario: ver sección "Videos complementarios" al final del documento (video 4).

El contrato social de los requerimientos — Derechos del usuario

- Esperar del analista que hable su lenguaje.
- Que el analista aprenda sobre su negocio y sus objetivos.
- Que el analista escriba una Especificación de Requerimientos de Software.
- Recibir explicaciones de los productos creados en el proceso de requerimientos.
- Esperar de los desarrolladores un trato respetuoso.
- Obtener de los analistas y desarrolladores ideas y alternativas para los requerimientos y su implementación.
- Describir características del producto que lo hagan simple y agradable de usar.

- Manejar opciones de ajustar sus requerimientos para permitir la reutilización de componentes de software preexistentes.
- Recibir estimaciones de buena fe de costos de los cambios.
- Recibir un sistema que satisfaga sus necesidades funcionales y de calidad.

El contrato social de los requerimientos — Deberes del usuario

- Educar a los analistas y desarrolladores en su negocio.
- Destinar tiempo a proveer y clarificar los negocios.
- Ser específico y preciso acerca de los requerimientos.
- Tomar decisiones a tiempo.
- Respetar las evaluaciones de los desarrolladores de costo y factibilidad.
- Establecer prioridades a los requerimientos.
- Revisar los documentos de requerimientos y evaluar prototipos.
- Comunicar los cambios a los requerimientos.
- Seguir el proceso de cambios de la organización de desarrollo.
- Respetar el proceso de Ingeniería de Requerimientos que el analista utiliza.

Resumiendo: temas clave de la clase

- Las dificultades esenciales del software: la complejidad, conformidad, cambiabilidad e invisibilidad impactan en nuestro entendimiento de lo que nos piden.
- Problemática de la representación/modelo/descripción: diferenciación modelo-realidad; motivos para construir modelos (entender y describir lo que queremos construir); los modelos se insertan en la visión del desarrollo como secuencia de transformaciones; en el proceso de transformación necesitamos la abstracción.
- Dificultades para entender los requerimientos: en lo que se ve y en lo que se lee; es la parte más difícil de la construcción de un sistema software; en la obtención del conocimiento; en los usuarios, desarrolladores y su relación.
- Los stakeholders: visión amplia de los que impactan o son impactados por un sistema; heterogeneidad de los stakeholders; diferencias usuarios-analistas, gap semántico.
- El sesgo del punto de vista con el que "vemos" la realidad; los problemas que introduce la comunicación; el contrato social.

Con este último concepto se da por terminada la clase 2. Se sugiere ver los videos complementarios indicados a continuación para ampliar los conocimientos.

Videos complementarios

No fue posible acceder al contenido de estos videos para resumirlos (no están indexados ni son reproducibles desde acá). Se listan en el orden en que aparecen en el material, junto con el tema que amplían según su ubicación en el texto original. Si compartís la transcripción de cada uno, puedo reemplazar esta referencia por un resumen real.

Video 1 — Ubicado tras "Las dificultades esenciales del software": amplía la complejidad, conformidad, modificabilidad e invisibilidad del software.

<https://youtu.be/UvEcFGY-rBY?si=AlLpfazSiBow7s84>

Video 2 — Ubicado tras "El punto de vista del observador": amplía los conceptos de modelo, abstracción y transformación en el desarrollo de software.

<https://youtu.be/Cj3mwd3AFUo?si=xHLtD6kXH1rWFW6e>

Video 3 — Ubicado tras "La dificultad para obtener requerimientos": amplía el problema de adquirir conocimiento de los usuarios y otras fuentes.

<https://youtu.be/gzWuJuOcfBE?si=vJB-jo8B6qQGQOU3>

Video 4 — Ubicado tras "Gap semántico": amplía las diferencias de lenguaje y formación entre usuarios y analistas, y el contrato social de los requerimientos.

https://youtu.be/3p5JLfquT2o?si=BPr4TZSo-3_j8APb